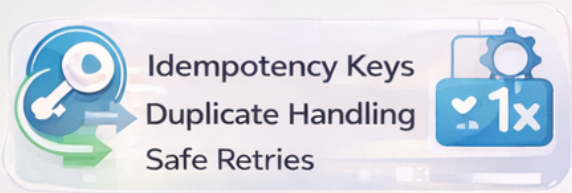
Day 6: Coordination, Idempotency & Distributed Workflows

Reliability & Retries

- Timeouts & Retries
- Exponential Backoff
- Jitter

Idempotency & Deduplication



- Idempotency Keys
- Duplicate Handling
- Safe Retries

Coordination

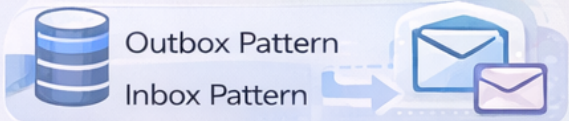
- Distributed Locks
- Leases
- Leader Election

- At-Most-Once
- At-Least-Once
- Exactly-Once (Trade-Offs)

Distributed Transactions

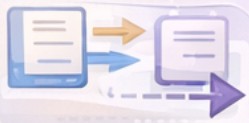
- Saga Pattern
- Compensating Actions

Data Consistency Patterns



Distributed Transactions

- Saga Pattern
- Leader Election



Workflow Management

- Orchestration
- Choreography

Reliability and security patterns make distributed systems **scalable, resilient, and safe.**

Day 6: Coordination, Idempotency, and Distributed Workflows

Day 6 focuses on how distributed systems stay correct when requests are retried, events are duplicated, and multi-service workflows fail halfway through.

It covers:

- timeouts, retries, exponential backoff, and jitter
- idempotency and deduplication
- at-most-once, at-least-once, and the "exactly-once" trade-off
- distributed locks, leases, and leader election
- sagas and compensating transactions
- outbox and inbox patterns
- workflow orchestration vs choreography

In short, Day 6 is about protecting business correctness when a system is no longer a single transaction on a single machine.

Core ideas

Timeouts, retries, and backoff

- `Timeouts` stop resources from being blocked forever by slow dependencies.
- `Retries` improve success rate for transient failures, but they can amplify load if done carelessly.
- `Exponential backoff` and `jitter` reduce retry storms and make recovery smoother under failure.

Idempotency and deduplication

- `Idempotency` means repeating the same request does not create a different business outcome.
- This matters for payments, order creation, webhook delivery, and message consumption.
- A common implementation stores the result of an `idempotency key` and returns the saved response on duplicate requests.

Delivery guarantees

- `At-most-once` avoids duplicates, but can lose work.
- `At-least-once` avoids loss, but consumers must handle duplicates.
- `Exactly-once` is usually only achievable within narrow boundaries and still depends on idempotent application logic.

Locks, leases, and leader election

- `Distributed locks` coordinate access to shared resources across instances.

- `Leases` are time-bound locks that reduce permanent deadlock risk when an owner crashes.
- `Leader election` picks one active coordinator for tasks like scheduling, partition ownership, or failover management.

Sagas, outbox, and workflow control

- A `saga` breaks one large distributed transaction into smaller local transactions with compensating actions.
- The `outbox pattern` writes business state and pending events together so events are not lost between database and broker.
- `Orchestration` centralizes workflow control, while `choreography` lets services react to events independently.

Day-6 sample: Idempotent Request Processor

This repository includes a small idempotency-key sample in both Python and Java.

Functional requirements

- Accept a request with an idempotency key
- Process a new request exactly once at the application level
- Return the stored response when the same key is retried
- Keep the implementation simple enough for interview explanation

High-level components

- `Client`: sends a request and retry key
- `API/service handler`: checks whether the key was already seen
- `Idempotency store`: keeps the response for processed keys
- `Business logic`: performs the actual action only for new keys

Data flow

1. A client sends a request with an idempotency key such as `payment-req-101`
2. The service checks whether that key already exists in the idempotency store
3. If the key is new, the business action runs and its response is stored
4. If the same request is retried, the stored response is returned
5. The client sees a stable result even if the network caused duplicate submissions

Project structure

```
Day-6/
  README.md
  idempotency-sample/
    python/
      idempotency_store.py
      demo.py
```

```
  java/
    IdempotencyStore.java
    Main.java
```

Run the sample

Python

```
cd idempotency-sample\python
python demo.py
```

Java

```
cd idempotency-sample\java
javac Main.java IdempotencyStore.java
java Main
```

Interview takeaways

- Retries are only safe when the write path is idempotent
- "Exactly-once" usually turns into at-least-once delivery plus deduplication or idempotent consumers
- Distributed locks solve coordination problems, but they also introduce failure and expiry edge cases
- Sagas are often more practical than distributed two-phase commit in service-oriented architectures
- Outbox and inbox patterns are common when events must stay consistent with database state

Next improvements

- Track `in-progress` requests separately from completed ones
- Add response status codes and error caching rules
- Add request payload hashing to reject key reuse with different payloads
- Persist keys in Redis or a database with TTL support
- Extend the sample into a payment or order creation API

Source Code Appendix

Day-6/idempotency-sample/python/idempotency_store.py

```
from __future__ import annotations

from dataclasses import dataclass
from threading import Lock
from typing import Callable, Dict, Generic, TypeVar

T = TypeVar("T")


@dataclass(frozen=True)
class ProcessResult(Generic[T]):
    value: T
    replayed: bool


class IdempotencyStore(Generic[T]):
    def __init__(self) -> None:
        self._responses: Dict[str, T] = {}
        self._lock = Lock()

    def execute(self, key: str, handler: Callable[[], T]) -> ProcessResult[T]:
        if not key:
            raise ValueError("key must be non-empty")

        with self._lock:
            if key in self._responses:
                return ProcessResult(value=self._responses[key], replayed=True)

            value = handler()
            self._responses[key] = value
            return ProcessResult(value=value, replayed=False)

    def processed_keys(self) -> int:
        with self._lock:
            return len(self._responses)
```

Day-6/idempotency-sample/python/demo.py

```
from idempotency_store import IdempotencyStore


class PaymentService:
    def __init__(self) -> None:
        self._processed_count = 0

    def charge(self, customer_id: str, amount_cents: int) -> str:
        self._processed_count += 1
        payment_id = f"pay-{self._processed_count:04d}"
        return (
            f"payment_id={payment_id}, customer_id={customer_id}, "
            f"amount_cents={amount_cents}"
        )

    @property
    def processed_count(self) -> int:
        return self._processed_count


def main() -> None:
    store = IdempotencyStore[str]()
    payments = PaymentService()

    attempts = [
        ("payment-req-101", "cust-7", 2500),
        ("payment-req-101", "cust-7", 2500),
        ("payment-req-102", "cust-7", 4100),
        ("payment-req-102", "cust-7", 4100),
    ]

    for key, customer_id, amount_cents in attempts:
        result = store.execute(
            key,
```

```
            lambda customer_id=customer_id, amount_cents=amount_cents: payments.charge(
                customer_id, amount_cents
            ),
        )
        status = "replayed" if result.replayed else "processed"
        print(f"{key}: {status} -> {result.value}")

    print()
    print("Business operations executed:", payments.processed_count)
    print("Stored idempotency keys:", store.processed_keys())


if __name__ == "__main__":
    main()
```

Day-6/idempotency-sample/java/IdempotencyStore.java

```
import java.util.HashMap;
import java.util.Map;
import java.util.function.Supplier;

public class IdempotencyStore<T> {
    private final Map<String, T> responses = new HashMap<>();

    public synchronized ProcessResult<T> execute(String key, Supplier<T> handler) {
        if (key == null || key.isBlank()) {
            throw new IllegalArgumentException("key must be non-empty");
        }

        if (responses.containsKey(key)) {
            return new ProcessResult<>(responses.get(key), true);
        }

        T value = handler.get();
        responses.put(key, value);
        return new ProcessResult<>(value, false);
    }

    public synchronized int processedKeys() {
        return responses.size();
    }

    public record ProcessResult<T>(T value, boolean replayed) {
    }
}
```

Day-6/idempotency-sample/java/Main.java

```
public class Main {
    public static void main(String[] args) {
        IdempotencyStore<String> store = new IdempotencyStore<>();
        PaymentService payments = new PaymentService();

        processAttempt(store, payments, "payment-req-101", "cust-7", 2500);
        processAttempt(store, payments, "payment-req-101", "cust-7", 2500);
        processAttempt(store, payments, "payment-req-102", "cust-7", 4100);
        processAttempt(store, payments, "payment-req-102", "cust-7", 4100);

        System.out.println();
        System.out.println("Business operations executed: " + payments.getProcessedCount());
        System.out.println("Stored idempotency keys: " + store.processedKeys());
    }

    private static void processAttempt(
            IdempotencyStore<String> store,
            PaymentService payments,
            String key,
            String customerId,
            int amountCents
    ) {
        IdempotencyStore.ProcessResult<String> result =
                store.execute(key, () -> payments.charge(customerId, amountCents));

        String status = result.replayed() ? "replayed" : "processed";
        System.out.printf("%s: %s -> %s%n", key, status, result.value());
    }
```

```
    private static final class PaymentService {
        private int processedCount = 0;

        public String charge(String customerId, int amountCents) {
            processedCount++;
            String paymentId = String.format("pay-%04d", processedCount);
            return String.format(
                    "payment_id=%s, customer_id=%s, amount_cents=%d",
                    paymentId,
                    customerId,
                    amountCents
            );
        }

        public int getProcessedCount() {
            return processedCount;
        }
    }
}
```